BỘ KHOA HỌC CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



HỆ THỐNG ÔN LUYỆN THI TỐT NGHIỆP TRUNG HỌC PHỔ THÔNG

Người hướng dẫn : MSc. Ngô Văn Quyền

Họ và tên sinh viên : Trần Hoàng Thiên - B23DCCN784

Phùng Văn Dương - B23DCCN226

Học Phần : Lập trình mạng

Chuyên ngành : Công nghệ phần mềm

Khoa : Công nghệ thông tin

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên *(ký và ghi rõ họ tên)*: .....

TABLE OF CONTENTS

HỆ THỐNG ÔN LUYỆN THI TRUNG HỌC PHỔ THÔNG.....1
I. TÓM TẮT NỘI DUNG 1
II. MỞ ĐẦU VÀ TỔNG QUAN.....1
2.1. Bối cảnh 1
2.1. Vấn đề 2
2.2. Động lực.....2
2.3. Phát biểu bài toán.....3
2.4. Mục tiêu 3
2.5. Phạm vi.....3
2.6. Đóng góp chính.....4
2.7. Kiến thức nền tảng 4
2.8. Các giải pháp liên quan và hạn chế.....5
2.9. Giải pháp khắc phục của dự án 5
III. PHÂN TÍCH VÀ ĐẶC TẢ BÀI TOÁN 6
3.1. Tổng quan bài toán.....6
3.2. Dữ liệu đầu vào 6
3.3. Dữ liệu đầu ra.....7
3.4. Ràng buộc.....7
3.5. Giả định.....7
3.6. Mô hình mạng 8
3.7. Yêu cầu hệ thống.....8
3.8. Liên hệ với các ca sử dụng.....8
IV. THIẾT KẾ HỆ THỐNG VÀ GIẢI PHÁP 9
4.1. Giải pháp đề xuất 9
4.2. Kiến trúc hệ thống.....14
4.3. Thiết kế giao tiếp mạng.....18
V. TRIỂN KHAI VÀ THỰC NGHIỆM 23
5.1. Môi trường thực nghiệm 23

5.2. Kiểm thử chức năng 23
5.3. Kiểm thử tải.....25
5.4. 5 Phân tích kết quả 26
VI ĐÓNG GÓP, HẠN CHẾ VÀ KẾT LUẬN.....27
6.1 ĐÓNG GÓP VÀ TÍNH MỚI CỦA ĐỀ TÀI.....27
6.2. Hạn chế hiện tại và Hướng phát triển.....28
6.3. Kết luận.....28
TÀI LIỆU THAM KHẢO 29
PHỤ LỤC.....29
Phụ lục A: Schema cơ sở dữ liệu rút gọn 30
Phụ lục B: Cấu hình môi trường (.env mẫu).....30
Phụ lục C: Tóm tắt kịch bản kiểm thử tải k6 (exams.js).....31

HỆ THỐNG ÔN LUYỆN THI TRUNG HỌC PHỔ THÔNG

I. TÓM TẮT NỘI DUNG

Hiện nay, học sinh ôn thi tốt nghiệp THPT thường thiếu một nền tảng tập trung đáp ứng đồng thời các nhu cầu luyện đề đa dạng, tương tác hỏi đáp nhóm, tự soạn đề cá nhân và bảo mật dữ liệu bài làm. Để giải quyết vấn đề này, đề tài xây dựng một ứng dụng web ôn luyện thi trực tuyến toàn diện theo mô hình MVC sử dụng công nghệ Node.js, Express, Pug và MongoDB, tích hợp hệ thống xác thực bảo mật bằng JWT cookie, phân quyền người dùng (User/Admin) và hệ thống thông báo thời gian thực qua Socket.IO. Về mặt triển khai, hệ thống hỗ trợ ba loại câu hỏi (trắc nghiệm, đúng/sai, trả lời ngắn), module soạn thảo câu hỏi kết hợp công thức LaTeX và tải ảnh lên Cloudinary. Đặc biệt, hệ thống ứng dụng cơ chế snapshot dữ liệu nhằm cố định nội dung câu hỏi tại thời điểm bắt đầu làm bài, kết hợp điều kiện cập nhật nguyên tử trực tiếp trên MongoDB để kiểm soát việc nộp bài đồng thời và tránh ghi đè kết quả. Kết quả chính của đề tài là một hệ thống hoạt động ổn định, cung cấp đầy đủ các tính năng chấm điểm tự động, quản lý ngân hàng câu hỏi, thông báo realtime và đã được kiểm chứng khả năng chịu tải thực tế thông qua công cụ k6 với các mức 10, 50 và 100 VUs. Cuối cùng, dự án mang lại đóng góp thiết thực khi cung cấp một giải pháp công nghệ giáo dục hiệu quả cho kỳ thi tốt nghiệp THPT, đồng thời khẳng định giá trị kỹ thuật thông qua giải pháp bảo toàn lịch sử bài làm bằng snapshot và cơ chế xử lý nộp bài đồng thời an toàn, chính xác.

Keywords : Node.js; Hệ thống ôn thi THPT; Mô hình MVC; Socket.IO; Cơ chế Snapshot.

II. MỞ ĐẦU VÀ TỔNG QUAN

2.1. Bối cảnh

Kỳ thi tốt nghiệp Trung học Phổ thông (THPT) quốc gia luôn là một cột mốc quan trọng, đánh giá toàn diện quá trình học tập phổ thông của học sinh và là căn cứ cốt

lõi để xét tuyển vào các trường đại học, cao đẳng. Trong bối cảnh chuyển đổi số giáo dục diễn ra mạnh mẽ, các nền tảng học tập và ôn luyện trực tuyến đã trở thành công cụ không thể thiếu, hỗ trợ người học tiếp cận nguồn tài nguyên phong phú mọi lúc, mọi nơi. Tuy nhiên, việc tối ưu hóa trải nghiệm tự học, cá nhân hóa quá trình ôn tập, kết hợp tương tác nhóm và đảm bảo tính toàn vẹn dữ liệu bài làm trong môi trường thi trực tuyến vẫn đang là những bài toán lớn đặt ra cho các nhà phát triển hệ thống giáo dục hiện đại.

2.1. Vấn đề

Thực tế quá trình ôn luyện của học sinh phổ thông hiện nay đang gặp phải nhiều rào cản đáng chú ý. Trước hết là sự phân mảnh về mặt tài nguyên, khi các đề thi và tài liệu tham khảo thường nằm rải rác trên các diễn đàn hoặc mạng xã hội, thiếu đi một hệ thống quản lý và lưu trữ tập trung được chuẩn hóa theo môn học và chuyên đề. Bên cạnh đó, các công cụ hiện hành còn hạn chế về tính tương tác và tính tự chủ, khiến học sinh thiếu không gian để chủ động soạn thảo câu hỏi cá nhân, chia sẻ đề thi trong các nhóm học tập hoặc trao đổi thắc mắc theo thời gian thực. Đáng lo ngại hơn cả là rủi ro về tính toàn vẹn dữ liệu trong các hệ thống thi trực tuyến thông thường: khi câu hỏi gốc bị thay đổi hoặc chỉnh sửa bởi quản trị viên, lịch sử bài làm và kết quả đã lưu của học sinh thường bị thay đổi theo hoặc bị ghi đè, làm mất đi tính chính xác của dữ liệu đánh giá lịch sử.

2.2. Động lực

Từ những bất cập trên, động lực của nghiên cứu xuất phát từ nhu cầu thiết yếu là xây dựng một giải pháp công nghệ toàn diện, mã nguồn mở, vừa cung cấp môi trường ôn luyện thi THPT chuẩn mực, vừa giải quyết triệt để các thách thức kỹ thuật cốt lõi trong lập trình web hiện đại. Dự án tập trung khai thác sức mạnh của hệ sinh thái Node.js kết hợp mô hình kiến trúc MVC, cơ chế giao tiếp hai chiều thời gian thực qua Socket.IO và giải pháp bảo toàn dữ liệu thông qua cơ chế snapshot, nhằm mang lại một nền tảng vận hành ổn định, minh bạch và có tính ứng dụng thực tiễn cao.

2.3. Phát biểu bài toán

Đề tài tiến hành xây dựng một hệ thống website ôn luyện thi tốt nghiệp THPT toàn diện mang tên THPT Practice. Hệ thống được thiết kế nhằm đảm bảo khả năng phân quyền chặt chẽ giữa người dùng thông thường và quản trị viên thông qua phương thức xác thực bảo mật bằng JWT cookie. Đồng thời, nền tảng hỗ trợ học sinh thực hiện bài thi với nhiều định dạng câu hỏi phong phú như trắc nghiệm, đúng/sai và trả lời ngắn, cho phép tự soạn đề thi, tích hợp hiển thị công thức toán học KaTeX và tải ảnh minh họa qua Cloudinary. Ngoài ra, ứng dụng còn thúc đẩy mô hình học tập cộng tác thông qua tính năng tạo nhóm, chia sẻ đề và hệ thống thông báo realtime, đồng thời bảo vệ tuyệt đối dữ liệu bài làm bằng cơ chế snapshot độc lập kết hợp thuật toán kiểm soát nộp bài đồng thời trên cơ sở dữ liệu MongoDB.

2.4. Mục tiêu

Mục tiêu tổng quát của đề tài là thiết kế, phát triển và thử nghiệm thành công nền tảng thpt-practice phục vụ hiệu quả cho nhu cầu ôn thi THPT trực tuyến. Để đạt được mục tiêu này, các nhiệm vụ cụ thể được đặt ra bao gồm việc xây dựng giao diện người dùng trực quan, render phía server bằng Pug kết hợp tối ưu hóa CSS và JavaScript. Hệ thống cần hoàn thiện trọn vẹn các module quản trị ngân hàng câu hỏi, môn học, chuyên đề và đề thi, đồng thời triển khai thành công cơ chế snapshot dữ liệu bài làm cùng thuật toán kiểm soát luồng nộp bài đồng thời dạng atomic update. Cuối cùng, hiệu năng thực tế của hệ thống sẽ được kiểm chứng và đánh giá định lượng thông qua công cụ đo tải k6 ở các mức áp lực 10, 50 và 100 người dùng ảo đồng thời (VUs).

2.5. Phạm vi

Phạm vi nghiên cứu và triển khai của đề tài bao gồm cả khía cạnh chức năng lẫn kỹ thuật. Về mặt chức năng, hệ thống tập trung hoàn thiện các tính năng quản lý tài khoản, phân quyền, ngân hàng câu hỏi đa dạng định dạng, tổ chức thi trực tuyến, chấm điểm tự động, quản lý nhóm học tập, gửi thông báo thời gian thực và cung cấp

bảng điều khiển dành riêng cho quản trị viên. Về mặt kỹ thuật, dự án được phát triển trên nền tảng Node.js (từ phiên bản 20 trở lên) kết hợp framework Express 5, cơ sở dữ liệu MongoDB thông qua Mongoose ORM, kết hợp các thư viện phụ trợ như Socket.IO, KaTeX, Cloudinary và thực hiện đánh giá kiểm thử tải bằng k6 trên môi trường thử nghiệm nội bộ.

2.6. Đóng góp chính

Đóng góp của nghiên cứu thể hiện qua hai khía cạnh thực tiễn và kỹ thuật. Về mặt thực tiễn, đề tài cung cấp một ứng dụng web giáo dục hoàn chỉnh, hỗ trợ học sinh nâng cao hiệu quả tự học và giúp giáo viên quản lý ngân hàng câu hỏi một cách hệ thống. Về mặt kỹ thuật, đề tài đóng góp hai giải pháp cốt lõi: một là đề xuất và hiện thực hóa cơ chế snapshot dữ liệu bài làm, giúp bảo toàn vẹn toàn lịch sử kết quả của học sinh trước mọi thay đổi từ ngân hàng câu hỏi gốc; hai là xây dựng cơ chế kiểm soát nộp bài đồng thời dựa trên điều kiện nguyên tử của MongoDB, triệt tiêu hoàn toàn tình trạng xung đột và ghi đè kết quả khi có nhiều luồng request nộp bài diễn ra sát nhau.

2.7. Kiến thức nền tảng

Việc phát triển hệ thống thpt-practice dựa trên các nền tảng kiến thức và công nghệ cốt lõi của lập trình web hiện đại. Kiến trúc hệ thống tuân theo mô hình MVC (Model - View - Controller), cho phép tách biệt rõ ràng giữa logic dữ liệu, giao diện hiển thị qua template engine Pug và luồng điều hướng xử lý nghiệp vụ trên Express, từ đó giúp mã nguồn đạt tính đóng gói cao, dễ bảo trì và mở rộng. Đối với cơ chế xác thực, hệ thống ứng dụng JSON Web Token (JWT) lưu trữ trong HTTP-only cookie kết hợp các middleware bảo mật, mang lại cơ chế quản lý phiên đăng nhập và phân quyền User/Admin an toàn, hạn chế tối đa nguy cơ bị tấn công XSS. Nhằm đáp ứng yêu cầu tương tác thời gian thực, giao thức WebSocket thông qua Socket.IO được triển khai trên cùng server HTTP để duy trì kết nối hai chiều bền vững, cho phép máy chủ chủ động đẩy các thông báo đến trình duyệt ngay lập tức mà không tốn chi phí thăm dò

lặp lại. Cuối cùng, khả năng xử lý đồng thời của hệ thống được tối ưu hóa nhờ vào mô hình I/O bất đồng bộ (Non-blocking I/O) đơn luồng của Node.js, giúp phục vụ lượng lớn yêu cầu từ người dùng cùng lúc mà không đòi hỏi chi phí khởi tạo luồng xử lý riêng biệt.

2.8. Các giải pháp liên quan và hạn chế

Nghiên cứu tiến hành khảo sát và phân tích các hệ thống khảo thí trực tuyến hiện có trên thị trường để làm rõ tính cấp thiết của đề tài, qua đó chia các giải pháp hiện nay thành hai nhóm chính với những hạn chế cụ thể. Nhóm thứ nhất bao gồm các nền tảng luyện thi thương mại độc quyền (mã nguồn đóng) như HocMai, Tuyensinh247, hoặc các tính năng trắc nghiệm trả phí trên hệ thống LMS của các trường phổ thông và đại học. Mặc dù sở hữu kho ngân hàng câu hỏi lớn, các nền tảng này lại áp dụng mô hình thương mại với chi phí truy cập cao, yêu cầu người dùng mua gói tài khoản theo năm hoặc theo môn. Đồng thời, việc sử dụng mã nguồn đóng hoàn toàn không cho phép học sinh hay giáo viên tự do tùy biến giao diện, tự soạn thảo ngân hàng câu hỏi cá nhân, hay chủ động thiết lập các nhóm học tập và phòng thi riêng theo nhu cầu ôn luyện.

Bên cạnh đó, Nhóm thứ hai gồm các ứng dụng thi trắc nghiệm mã nguồn mở cơ bản (self-hosted hoặc open-source) như các mã nguồn quản lý bài tập đơn giản xây dựng trên nền tảng PHP và MySQL truyền thống, hay các mẫu template Django và Node.js cơ bản trên GitHub, thường dựa trên liên kết khóa ngoại trực tiếp bằng ID của bảng câu hỏi. Nhóm giải pháp này bộc lộ những hạn chế kỹ thuật nghiêm trọng. Thứ nhất là tình trạng sai lệch lịch sử đánh giá năng lực do cấu trúc dữ liệu liên kết trực tiếp bảng lịch sử bài làm (`TestResults`) với ID câu hỏi (`Questions`); khi giáo viên chỉnh sửa nội dung hoặc đáp án trong hệ thống, các bài thi học sinh đã làm trong quá khứ bị thay đổi theo, làm mất đi tính toàn vẹn của dữ liệu khảo thí lịch sử. Thứ hai là lỗi xung đột dữ liệu (*race condition*) phát sinh do hệ thống thiếu cơ chế khóa trạng thái (*state locking*) và xử lý đồng thời, khiến việc nhiều học sinh cùng nộp bài thi vào thời điểm thời gian đếm ngược về 0 dễ dẫn đến hiện tượng ghi đè dữ liệu trên cơ sở dữ liệu.

2.9. Giải pháp khắc phục của dự án

Hệ thống thpt-practice được thiết kế để khắc phục triệt để những hạn chế nêu trên. Bằng cách tích hợp trình soạn thảo linh hoạt hỗ trợ công thức KaTeX và tải ảnh Cloudinary, đi kèm các tính năng nhóm học tập tương tác realtime qua Socket.IO, hệ thống mang đến một môi trường tự học chủ động và toàn diện. Đặc biệt, việc ứng dụng cơ chế snapshot bài làm kết hợp với cơ chế cập nhật nguyên tử (atomic update) trên MongoDB chính là lời giải kỹ thuật then chốt, đảm bảo tính chính xác, minh bạch và toàn vẹn dữ liệu tuyệt đối cho lịch sử làm bài của học sinh.

III. PHÂN TÍCH VÀ ĐẶC TẢ BÀI TOÁN

3.1. Tổng quan bài toán

Hệ thống THPT Practice là một ứng dụng web trực tuyến nhằm tự động hóa toàn diện quy trình ôn luyện, kiểm tra kiến thức và quản lý ngân hàng câu hỏi phục vụ kỳ thi tốt nghiệp THPT. Hệ thống phục vụ đồng thời hai nhóm tác nhân chính là người học và quản trị viên. Trong đó, người học có thể làm đề, luyện tập, tự soạn câu hỏi và đề thi, tham gia các nhóm học tập. Quản trị viên chịu trách nhiệm quản lý nội dung hệ thống và người dùng.

3.2. Dữ liệu đầu vào

Hệ thống tiếp nhận trước hết là thông tin tài khoản phục vụ quá trình đăng ký và đăng nhập. Ngân hàng câu hỏi được tổ chức theo ba định dạng cốt lõi: câu hỏi trắc nghiệm khách quan, câu hỏi đúng/sai và câu hỏi trả lời ngắn yêu cầu điền đáp án. Nội dung câu hỏi có thể chứa công thức toán học được hiển thị bằng thư viện KaTeX, và cho phép đính kèm hình ảnh minh họa hợp lệ với dung lượng dưới 3 MB.

Bên cạnh đó, dữ liệu đầu vào còn gồm cấu trúc môn học, các chuyên đề ôn tập và bộ đề do người dùng tự soạn. Ngoài ra là các thao tác tương tác trực tuyến như làm bài thi, lựa chọn phương án trả lời, gửi yêu cầu tham gia nhóm học tập và các thông điệp truyền qua kết nối thời gian thực.

3.3. Dữ liệu đầu ra

Giao diện được kết xuất trực tiếp ở phía máy chủ bằng công cụ template Pug và hiển thị trên trình duyệt của người dùng. Trong quá trình học tập, hệ thống tự động chấm điểm và trả về kết quả chi tiết cho từng câu trả lời đúng hoặc sai. Lịch sử bài làm được lưu cố định bằng cơ chế bản sao dữ liệu độc lập (snapshot), nhờ đó kết quả cũ không bị ảnh hưởng khi nội dung đề gốc thay đổi.

Hệ thống cũng đẩy các thông báo sự kiện theo thời gian thực lên giao diện của người dùng. Riêng quản trị viên có thêm bảng điều khiển cung cấp thông tin tổng quan về hệ thống.

3.4. Ràng buộc

Về mặt kỹ thuật, ứng dụng phải chạy trên Node.js từ phiên bản 20 trở lên và sử dụng hệ quản trị cơ sở dữ liệu MongoDB. Hình ảnh tải lên thông qua dịch vụ Cloudinary phải vượt qua bước kiểm tra định dạng và giới hạn dung lượng để bảo vệ hiệu năng máy chủ.

Về mặt bảo mật, phiên làm việc và thông tin định danh được quản lý bằng JSON Web Token lưu trong HTTP-only cookie nhằm ngăn chặn nguy cơ chiếm đoạt phiên. Quyền truy cập vào các đường dẫn quản trị được phân định rõ ràng: mọi tài khoản không có vai trò quản trị viên đều bị từ chối.

Về toàn vẹn dữ liệu, để xử lý tình huống nhiều yêu cầu nộp bài đồng thời, hệ thống áp dụng các điều kiện cập nhật nguyên tử trực tiếp trên cơ sở dữ liệu. Cách này loại bỏ xung đột dữ liệu và hiện tượng ghi đè kết quả bài làm.

3.5. Giả định

Hệ thống vận hành dựa trên ba giả định nền tảng. Thứ nhất, môi trường triển khai luôn duy trì kết nối ổn định với cơ sở dữ liệu MongoDB. Thứ hai, trình duyệt của

người dùng hỗ trợ các chuẩn công nghệ web hiện đại, từ JavaScript ES6 trở lên. Thứ ba, các tệp cấu hình môi trường chứa các khóa bí mật hợp lệ.

3.6. Mô hình mạng

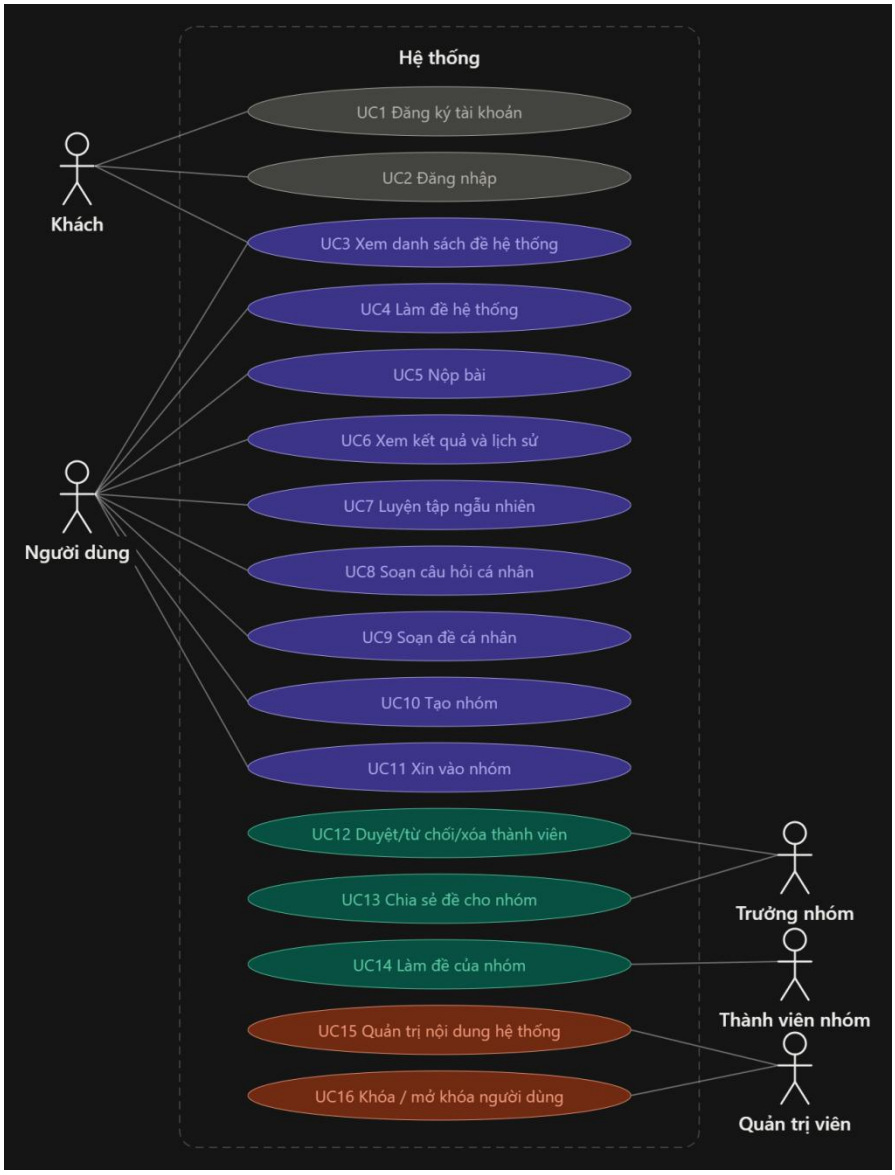
Mô hình mạng kết hợp hai giao thức. Giao thức HTTP tiêu chuẩn đảm nhận các yêu cầu truyền dữ liệu thông thường. Giao thức WebSocket, thông qua thư viện Socket.IO, duy trì một kết nối hai chiều bền vững. Nhờ đó máy chủ có thể chủ động đẩy thông báo thời gian thực đến giao diện người dùng mà không cần cơ chế thăm dò (polling) lặp lại.

3.7. Yêu cầu hệ thống

Hệ thống đòi hỏi sự đồng bộ giữa các thành phần phần mềm nền tảng. Cụ thể gồm Node.js (phiên bản 20 trở lên) làm môi trường thực thi và MongoDB làm cơ sở dữ liệu. Các thư viện chính gồm Express (máy chủ web), Mongoose (tương tác cơ sở dữ liệu), Pug (template), Socket.IO (thời gian thực), KaTeX (công thức toán) và Cloudinary (lưu trữ ảnh). Công cụ kiểm thử tải k6 được dùng để đánh giá hiệu năng, cùng với các công cụ hỗ trợ xử lý trung gian (middleware) khác.

3.8. Liên hệ với các ca sử dụng

Sơ đồ UC chung



IV. THIẾT KẾ HỆ THỐNG VÀ GIẢI PHÁP

4.1. Giải pháp đề xuất

4.1.1. Ý tưởng chung

Giải pháp xoay quanh một nguyên tắc: máy chủ là nơi duy nhất quyết định kết quả. Trình duyệt chỉ hiển thị nội dung, đếm giờ và gửi đáp án. Việc quan trọng đều do máy

chủ làm: chấm điểm, kiểm tra quyền, đổi trạng thái bài làm, duyệt thành viên. Hai cơ chế bổ trợ giữ dữ liệu đáng tin khi có nhiều người dùng cùng lúc:

- Bản chụp (snapshot): khi bắt đầu làm bài, câu hỏi được sao chép vào bài làm, nên điểm và lời giải không đổi dù câu hỏi gốc bị sửa.
- Cập nhật có điều kiện: một thao tác đổi trạng thái chỉ thành công khi bản ghi còn ở đúng trạng thái trước đó, nên thao tác lặp lại bị từ chối thay vì ghi trùng.

4.1.2. Kiến trúc MVC

Hệ thống theo mô hình MVC, giao diện dựng sẵn ở máy chủ (server-side rendering).

hành phần	**Vị trí**	**Quy mô**	**Vai trò**
Model	models/	9 tệp, 735 dòng	Định nghĩa dữ liệu và ràng buộc bằng Mongoose
View	views/, public/	Pug 7.070 dòng; JS 7.446 dòng	Dựng trang; JavaScript phía trình duyệt lo đồng hồ, trình soạn thảo, thông báo
Controller	controllers/	20 tệp, 3.945 dòng	Nhận yêu cầu, xử lý nghiệp vụ, truy vấn, trả trang hoặc JSON

Middleware	middlewares/	254 dòng	Xác thực, phân quyền, tải ảnh

Tầng nghiệp vụ chưa tách riêng: logic chấm điểm và truy vấn nằm trực tiếp trong controller. Cách này đơn giản, phù hợp quy mô hiện tại

4.1.3. Các quyết định thiết kế

Mã	**Quyết định**	**Lý do (suy ra)**	**Đánh đổi**	**Căn cứ**
D1	Ứng dụng nguyên khối, giao diện dựng ở máy chủ	Triển khai đơn giản, ít thành phần vận hành	Khó mở rộng ngang	index.js
D2	Chụp câu hỏi vào bài làm	Điểm và lời giải không đổi khi câu gốc bị sửa hoặc xóa	Tốn thêm dung lượng	Attempt.questions
D3	Mã băm SHA-256 của nội dung hiển thị, cho đề nhóm	Phát hiện đề đã đổi giữa lúc xem và lúc bắt	Phải tải lại trang; đề hệ thống chưa có	group-exam-taking.controller.js:86

		đầu (trả 409)	kiểm tra này	
D4	Chỉ gửi nội dung hiển thị xuống trình duyệt	Không lộ đáp án trước khi nộp	Cần trang kết quả riêng	displayQuestions
D5	findOneAndUpdate có điều kiện trạng thái	Chống nộp trùng, duyệt trùng, không cần giao dịch	Chỉ an toàn trong phạm vi một tài liệu	exam.controller.js:303-307
D6	Một mô hình cho đề và câu hỏi, phân biệt bằng scope, createdBy, groupIds	Tự soạn và chia sẻ trên cùng cấu trúc	Mọi truy vấn phải lọc đúng scope	question.model.js, exam.model.js
D7	Lưu thông báo trước, phát sau	Người ngoại tuyến vẫn thấy khi quay lại	Hai bước không nguyên tử	notification.service.js:11,19 (có chú thích)

D8	Máy chủ tự gán phòng account:<id>	Client không nghe được thông báo của người khác	Phụ thuộc cookie hợp lệ	sockets/index.js:51 (có chú thích)

4.1.4. Thuật toán chính

Chấm điểm khi nộp bài. Điểm tối đa của mỗi câu bằng 10 chia cho số câu.

mcq: đúng khi đáp án gửi lên trùng đáp án đúng; ngược lại 0
*truefalse: điểm = (số mệnh đề có giá trị true/false khớp) / (tổng số mệnh đề) * điểm tối đa của câu*
fill: đúng khi đáp án là chuỗi không rỗng và thuộc danh sách đáp án chấp nhận
tổng điểm = tổng điểm các câu, làm tròn 2 chữ số
findOneAndUpdate({_id, accountId, status: in_progress}, {answers, score, status: submitted})
không có bản ghi khớp -> trả 409

Câu đúng/sai có 4 mệnh đề nên mỗi mệnh đề đúng được 0,25 điểm của câu. Cách chấm này cần đối chiếu với quy chế chính thức của kỳ thi.

Tạo lượt luyện tập. Máy chủ kiểm tra số câu là số nguyên từ 1 đến 100, kiểm tra môn và chuyên đề đang hoạt động, rồi lấy mẫu ngẫu nhiên các câu hệ thống đã xuất bản bằng $sample. Nếu không đủ câu, yêu cầu bị từ chối kèm số câu hiện có. Lượt luyện tập được tạo ở trạng thái ready, thời lượng bằng 2 phút nhân số câu. Khi người học bấm bắt đầu, trạng thái chuyển sang in_progress bằng một cập nhật có điều kiện (practice.controller.js:79-81).

Xin vào nhóm và duyệt thành viên. Yêu cầu xin vào chỉ được ghi khi người xin không phải chủ nhóm và chưa nằm trong hàng chờ (group.controller.js:317-327). Duyệt thành viên chỉ chạy khi người gọi là chủ nhóm và người xin đang trong hàng chờ; hệ

thống thêm người này vào danh sách thành viên và xóa khỏi hàng chờ (dòng 455-463).

Chia sẻ đề. Đề được chia sẻ phải là đề cá nhân đã xuất bản của chính người chia sẻ và chưa được chia sẻ cho nhóm này. Hệ thống thêm nhóm vào groupIds bằng $addToSet, rồi gửi thông báo tới các thành viên bằng Promise.allSettled (exam-sharing.controller.js:31-32, 41).

4.1.5. Luồng dữ liệu

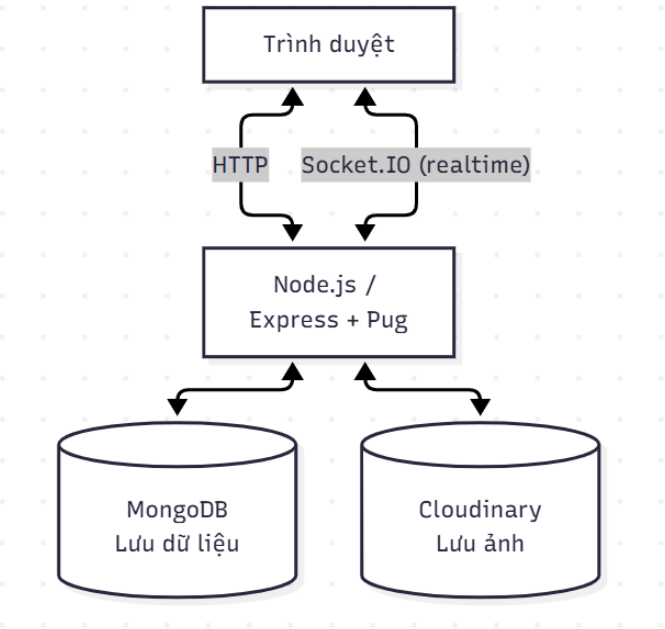
1. Trình duyệt gửi yêu cầu kèm cookie.
2. optionalAuth đọc cookie, kiểm tra token, nạp tài khoản đang hoạt động vào req.account.
3. Middleware của route (requireAuth, requireAdmin, requireMember) kiểm tra quyền.
4. Controller truy vấn Mongoose và xử lý nghiệp vụ.
5. Kết quả trả về là trang Pug, JSON hoặc chuyển hướng.
6. Với sự kiện của nhóm, controller gọi service thông báo: ghi vào MongoDB rồi phát tới phòng của người nhận.

4.2. **Kiến trúc hệ thống**

4.2.1. Tổng quan

Tầng	Thành phần	Giao tiếp
Client	Trình duyệt: HTML do Pug dựng sẵn; JS (đồng hồ, Quill + KaTeX, client Socket.IO)	HTTP/1.1; Socket.IO
Server	Node.js + Express 5: chuỗi middleware, routes / và /admin, controllers;	Mongoose; Cloudinary API

	Socket.IO dùng chung HTTP server	
Dữ liệu	MongoDB (9 collection)	Mongoose



4.2.2. Client

Trình duyệt nhận trang Pug đã dựng sẵn. JavaScript phía trình duyệt đảm nhiệm ba việc: đồng hồ làm bài (tự nộp khi hết giờ, thử nộp lại khi có mạng), trình soạn thảo câu hỏi (Quill và KaTeX), và nhận thông báo qua Socket.IO.

4.2.3. Server

Express 5 xử lý yêu cầu theo chuỗi middleware trong index.js:

Thứ tự	**Thành phần**	**Tác dụng**
1	morgan, compression	Ghi log, nén phản hồi

2	express.json, express.urlencoded (giới hạn 5 MB)	Đọc thân yêu cầu
3	cookieParser, optionalAuth	Đọc cookie, nạp tài khoản nếu có
4	express.static	Phục vụ tệp tĩnh
5	Route /admin, route /	Hai cây route tách biệt cho quản trị viên và người học
6	Xử lý 404 và 500	Trả văn bản thuần

Có hai điểm cần lưu ý. optionalAuth đứng trước express.static, nên yêu cầu tệp tĩnh có cookie sẽ kích hoạt một truy vấn tài khoản (nhận xét từ thứ tự mã, chưa đo). Và index.js gọi listen(3000) cố định dù đã khai báo hằng PORT.

4.2.4. Service và Socket.IO

Service duy nhất là thông báo: send() lưu thông báo rồi phát tới phòng của người nhận. Module sockets/ xác thực kết nối và gán phòng, như mô tả ở mục 3.3.2.

4.2.5. Database

MongoDB với 9 collection , Các quan hệ chính:

Quan hệ	**Cách biểu diễn**
Môn, chuyên đề, câu hỏi	Chuyên đề và câu hỏi tham chiếu subjectId; câu hỏi tham chiếu thêm topicId
Đề và câu hỏi	Đề giữ mảng questionIds

Đề và nhóm	Đề giữ mảng groupIds (nhóm được chia sẻ)
Nhóm và tài khoản	Nhóm giữ ownerId, mảng memberIds và pendingMemberIds
Bài làm	Nhúng bản chụp câu hỏi và đáp án; tham chiếu accountId, examId, groupId
Thông báo	Tham chiếu recipientId, senderId

4.2.6. Phân quyền

Mức	**Cách kiểm tra**	**Áp dụng**
Khách	Không cần token	Xem danh sách đề (/exams), nhóm (/groups)
Người dùng	requireAuth	Luyện tập, làm đề, soạn câu hỏi cá nhân
Quản trị viên	requireAdmin (role admin)	Toàn bộ cây /admin
Thành viên nhóm	requireMember	Xem và làm đề của nhóm; nhóm ngừng hoạt động trả 403
Chủ sở hữu	So sánh createdBy, ownerId hoặc accountId	Sửa, xóa tài nguyên của mình; xem kết quả bài làm của mình

Quản trị viên đăng nhập qua /auth/login như mọi người dùng. Tài khoản có role admin thấy liên kết “Trang quản trị” trên header, dẫn tới /admin/dashboard.

4.2.7. Cấu hình và triển khai

- **Yêu cầu:** Node.js từ 20.19 (do Mongoose 9.11.2 yêu cầu), MongoDB (local hoặc Atlas), tài khoản Cloudinary để tải ảnh.
- **Cấu hình:** tệp .env chứa MONGO_URI, JWT_SECRET và ba biến Cloudinary. Ứng dụng nghe cổng 3000.
- **Dữ liệu mẫu:** npm run seed tạo hai tài khoản (admin và user, mật khẩu băm bcrypt cost 10) và nạp dữ liệu từ scripts/data/. Seed idempotent: tài khoản đã có thì bỏ qua, các bản ghi dùng upsert với $setOnInsert.
- **Chạy:** npm start hoặc npm run dev (nodemon); giao diện được phục vụ cùng máy chủ, không cần chạy riêng.

4.3. Thiết kế giao tiếp mạng

4.3.1. Giao thức

- **HTTP/1.1** qua Express cho trang HTML (GET) và API JSON (POST, PATCH, DELETE). Tải ảnh dùng multipart/form-data. Ứng dụng không tự cấu hình TLS; mã hóa đường truyền phụ thuộc môi trường triển khai.
- **Socket.IO 4** gắn vào cùng máy chủ HTTP, khởi tạo bằng cấu hình mặc định (không khai báo CORS, nên chỉ nhận kết nối cùng nguồn). Theo tài liệu Socket.IO, mặc định kết nối bắt đầu bằng long-polling rồi nâng lên WebSocket khi trình duyệt hỗ trợ; điều này cần được xác nhận bằng quan sát thực tế. Máy chủ không lắng nghe sự kiện nào từ trình duyệt ngoài việc thiết lập kết nối, nên kênh được dùng một chiều, từ máy chủ xuống.

4.3.2. Kết nối và phiên làm việc

Sau khi đăng nhập thành công, máy chủ ký JWT chứa mã tài khoản và đặt vào cookie token (HttpOnly, SameSite=Lax, path /, sống 24 giờ). Mọi yêu cầu HTTP sau đó tự

mang cookie này. Khi mở kết nối Socket.IO, trình duyệt gửi cùng cookie trong lần bắt tay đầu tiên. Máy chủ thực hiện bốn bước:

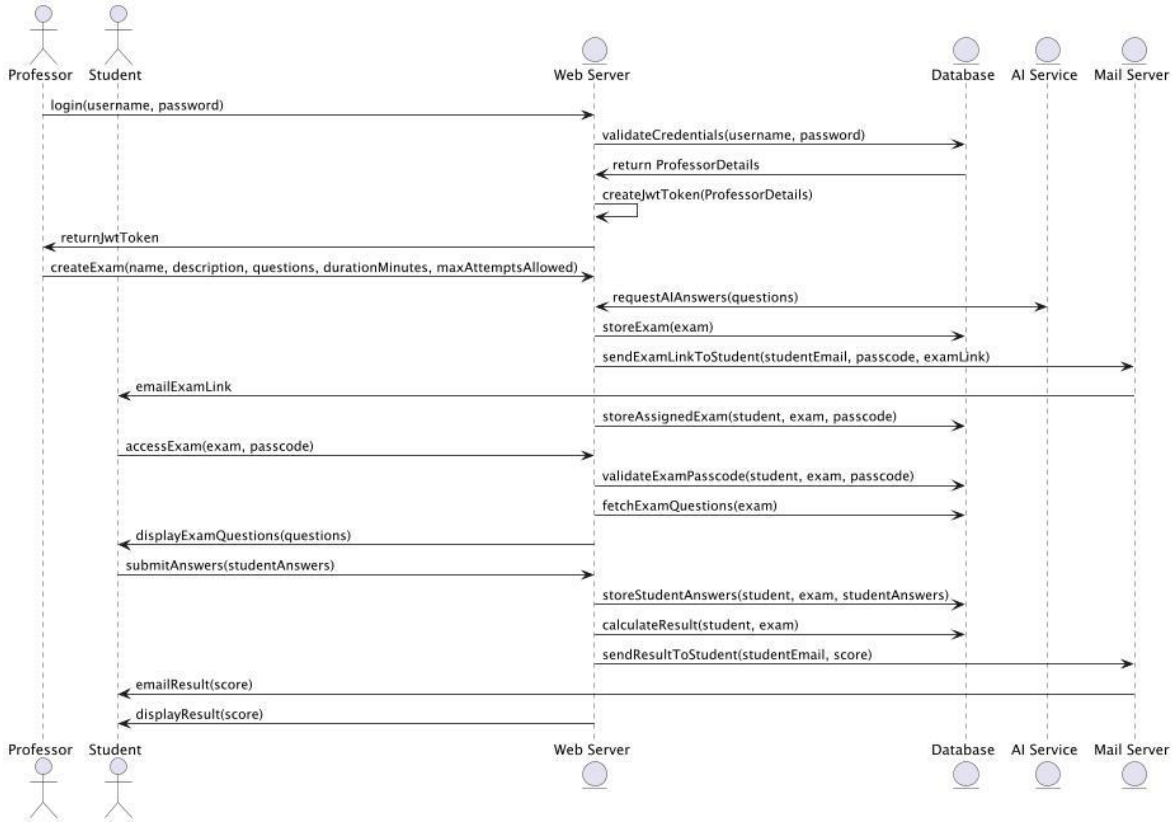
1. Đọc cookie và kiểm tra token.
2. Truy vấn tài khoản, bảo đảm tồn tại và đang hoạt động.
3. Ghi mã tài khoản vào kết nối.
4. Cho kết nối vào phòng account:<mã tài khoản>.

Nếu thất bại, kết nối bị từ chối kèm một trong ba thông điệp: “Vui lòng đăng nhập.”, “Token không hợp lệ.” hoặc “Tài khoản không khả dụng.”.

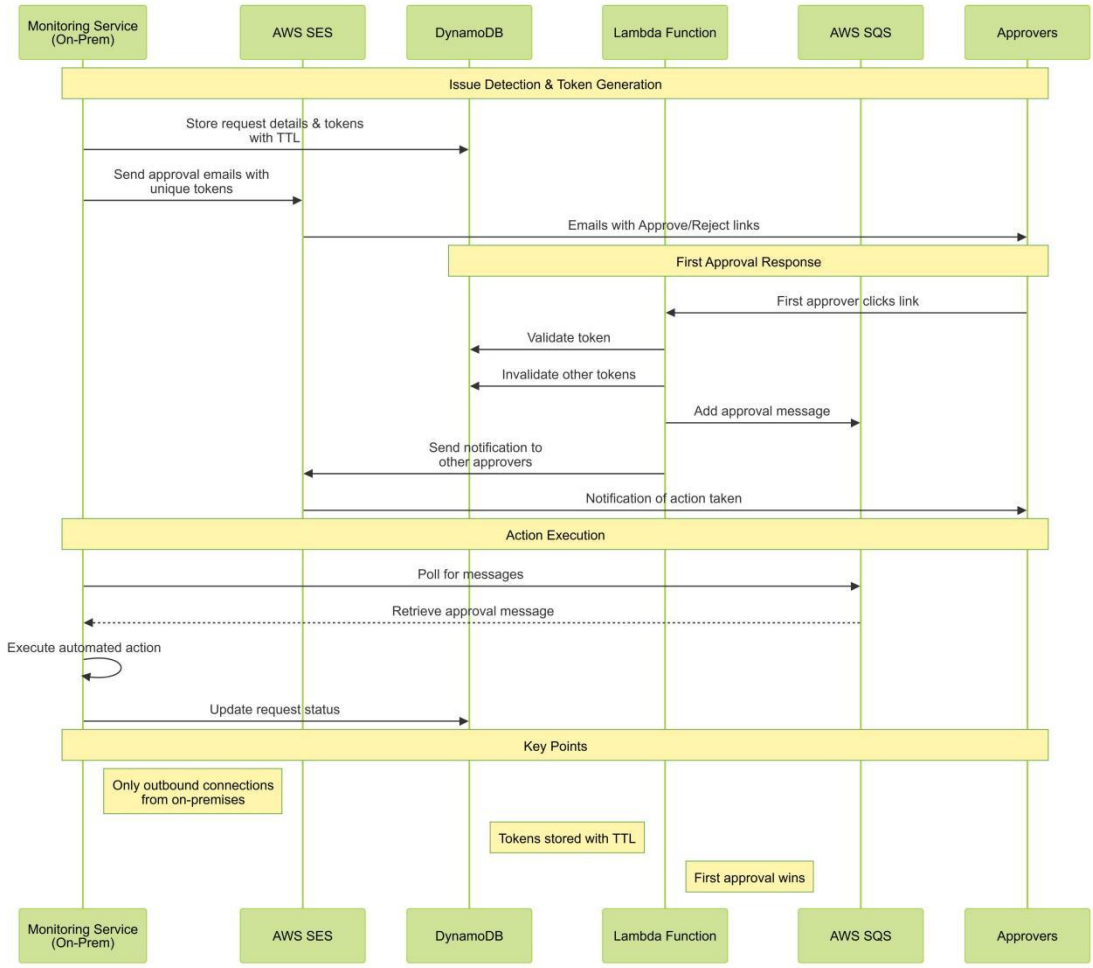
4.3.3. Định dạng thông điệp

Kiểu	**Khi nào**	**Ví dụ**
Trang HTML	Yêu cầu GET mở trang	Trang làm bài, kết quả
JSON	Yêu cầu từ JavaScript (có Accept: application/json)	Nộp bài, tạo luyện tập
Chuyển hướng	Biểu mẫu của nhóm	303 kèm ?notice=approved; riêng sửa nhóm dùng 302 kèm ?message=
Sự kiện Socket.IO	Có thông báo mới hoặc đã đọc	notification:new, notification:read

4.3.4. Trình tự yêu cầu và phản hồi



1. Trình duyệt gọi POST /exams/:examId/start. Máy chủ tải đề và kiểm tra từng câu hỏi (đã xuất bản, đúng phạm vi truy cập).
2. Máy chủ tạo bài làm kèm bản chụp câu hỏi và trả 201 với attemptId. Với đề nhóm, bước này còn so mã băm phiên bản.
3. Trình duyệt đếm giờ ở phía client; hết giờ thì tự nộp.
4. Khi nhận POST /exams/submit, máy chủ chấm theo bản chụp và cập nhật bài làm từ in_progress sang submitted. Nếu bài đã nộp, máy chủ trả 409.
5. Người học xem kết quả tại /exams/:attemptId/result.



Nhóm và thông báo

Học viên xin vào nhóm; máy chủ thêm vào hàng chờ, lưu thông báo cho chủ nhóm và phát notification:new tới phòng của chủ nhóm. Chủ nhóm duyệt; máy chủ chuyển học viên từ hàng chờ sang thành viên, lưu thông báo, phát cho học viên, rồi chuyển hướng 303 về trang quản lý.

Mã	Tình huống	Phản hồi
400	Thiếu answers, ID sai định dạng, số câu ngoài 1–100	JSON {code: "error", message}
401	Chưa đăng nhập	JSON nếu có Accept: application/json, ngược

		lại chuyển hướng /auth/login
403	Không phải admin, không phải thành viên, nhóm ngừng hoạt động	JSON hoặc văn bản thuần
404	Không thấy đề, nhóm, bài làm; đường dẫn không tồn tại	Văn bản hoặc JSON
409	Nộp lại bài đã nộp, đề đã đổi (mã băm lệch), câu hỏi trong đề không còn khả dụng, xin vào nhóm trùng	Không ghi dữ liệu
500	Lỗi không lường trước	Văn bản chung, chi tiết chỉ ghi log

Các cơ chế khác:

- Lỗi khi gửi thông báo được bắt và ghi log, không làm hỏng thao tác chính.
- Khi gửi nhiều thông báo song song, Promise.allSettled bảo đảm một người nhận lỗi không ảnh hưởng người khác.
- Nếu mất mạng lúc nộp, trình duyệt thử lại khi có mạng. Máy chủ đảm bảo một lượt không bị chấm hai lần (lần thứ hai nhận 409). Trình duyệt không có xử lý riêng cho mã 409, nên người học có thể thấy thông báo lỗi dù lần nộp đầu đã thành công; mình chưa kiểm chứng trải nghiệm này.
- Nếu MongoDB không kết nối được lúc khởi động, tiến trình thoát với mã 1.

V. TRIỂN KHAI VÀ THỰC NGHIỆM

5.1. Môi trường thực nghiệm

Hạng mục	Giá trị
CPU / RAM / ổ đĩa	Core i7 – 11800H / 16GB RAM / SSD 512GB
Hệ điều hành	Windows 11
Node.js / MongoDB / k6	v24.12.0 / v9.0.4 / v2.3.0
Vị trí MongoDB (Local)	mongodb://127.0.0.1:27017/THPT
Máy chạy k6 và máy chạy server	Cùng máy, kết nối qua http://localhost:3000
Số bản ghi dữ liệu mẫu (đề, câu hỏi, tài khoản)	10 đề thi ,60 câu hỏi , 2 tài khoản

5.2. Kiểm thử chức năng

Mười hai kịch bản dưới đây xuất phát từ các cơ chế đã thiết kế ở mục IV.

STT	Kịch bản	Kết quả mong đợi	Thực tế	Đánh giá
1	Đăng nhập đúng thông tin	Server đặt cookie token (HttpOnly, SameSite=Lax, 24 giờ)	Đăng nhập thành công; cookie token có đầy đủ thuộc tính và thời hạn yêu cầu	Đạt
2	Tài khoản User truy cập /admin/dashboard	403	Máy chủ trả HTTP 403, từ	Đạt

			chối truy cập trang quản trị	
3	Tạo câu hỏi mcq, truefalse, fill có công thức KaTeX	Lưu thành công; công thức hiển thị đúng	Tạo và lưu được cả ba dạng câu hỏi; công thức hiển thị đúng khi xem lại	Đạt
4	Tải ảnh ≥ 3 MB hoặc sai định dạng	Bị từ chối	Hệ thống từ chối ảnh vượt giới hạn dung lượng hoặc không đúng định dạng cho phép	Đạt
5	Tạo luyện tập với số câu = 0, 101 và số câu vượt kho	400; 400; từ chối kèm số câu hiện có	Hệ thống từ chối cả ba trường hợp và thông báo tương ứng	Đạt
6	POST /exams/:examId/start	201 kèm attemptId; bài làm chứa Snapshot	Máy chủ trả HTTP 201 và attemptId; bản ghi bài làm chứa dữ liệu snapshot câu hỏi	Đạt
7	Nộp bài với đáp án đúng, sai, một phần mệnh đề đúng/sai	Điểm tính theo mục 4.1.4, làm tròn 2 chữ số	Điểm hệ thống trùng với điểm tính thủ công; kết quả được làm tròn theo quy định	Đạt
8	Nộp lại bài đã nộp	409, điểm lưu không đổi	Máy chủ trả HTTP 409; kết quả đã lưu không bị thay đổi	Đạt

9	Sửa hoặc xóa câu hỏi gốc sau khi nộp, mở lại kết quả	Nội dung, đáp án, điểm không đổi	Trang kết quả vẫn hiển thị dữ liệu snapshot; nội dung, đáp án và điểm giữ nguyên	Đạt
10	Sửa đề nhóm giữa lúc mở trang và lúc bắt đầu	409	Máy chủ trả HTTP 409 và từ chối bắt đầu bài làm từ phiên bản đề cũ	Đạt
11	Xin vào nhóm, chủ nhóm duyệt	Chủ nhóm nhận notification:new; sau duyệt, học viên nhận thông báo; chuyển hướng 303	Chủ nhóm nhận thông báo yêu cầu tham gia; sau khi duyệt, học viên nhận thông báo chấp thuận; phản hồi chuyển hướng có mã HTTP 303	Đạt
12	Mở Socket.IO không cookie	Từ chối: “Vui lòng đăng nhập.”	Kết nối bị từ chối; nhận sự kiện connect_error với thông báo “Vui lòng đăng nhập.”	Đạt

5.3. **Kiểm thử tải**

Thực nghiệm sử dụng k6 để đánh giá khả năng đáp ứng của trang danh sách đề thi tại endpoint GET /exams. Kịch bản được thực hiện ở ba mức tải 10, 50 và 100 người dùng ảo (VUs), mỗi mức duy trì trong 60 giây. Sau thời gian này, k6 cho phép các vòng lặp đang thực hiện hoàn tất.

Trong mỗi vòng lặp, người dùng ảo gửi một yêu cầu, kiểm tra mã HTTP 200 và header Content-Type có chứa text/html, sau đó nghỉ 1 giây. Công cụ k6 và máy chủ ứng dụng chạy trên cùng máy tính.

Bảng 5.3. Kết quả kiểm thử tải đối với GET /exams

VUs	Số request	Req/s	p50 (ms)	p95 (ms)	Tỷ lệ lỗi (%)
10	580	9,56	37,04	49,20	0
50	1.706	27,63	770,90	886,00	0
100	1.739	27,16	2.580,00	3.060,00	0

5.4. 5 Phân tích kết quả

Khi tăng tải từ 10 lên 50 và 100 VUs, p95 tăng từ 49,20 ms lên 886 ms và 3.060 ms. Thông lượng gần như không tăng khi chuyển từ 50 lên 100 VUs, duy trì khoảng 27 yêu cầu/giây, cho thấy dấu hiệu bão hòa trong môi trường thử nghiệm.

Cả ba mức tải đều không ghi nhận yêu cầu HTTP thất bại và đạt toàn bộ điều kiện kiểm tra. Tuy nhiên, độ trễ tăng đáng kể ở mức tải cao. Do chưa đo tài nguyên và thời gian truy vấn, chưa đủ căn cứ xác định nguyên nhân giới hạn hiệu năng.

Phép thử chỉ đánh giá GET /exams trong môi trường cục bộ, với k6 và server chạy cùng máy; chưa đại diện cho hiệu năng toàn hệ thống, các thao tác ghi dữ liệu hoặc Socket.IO.

VI ĐÓNG GÓP, HẠN CHẾ VÀ KẾT LUẬN

6.1 ĐÓNG GÓP VÀ TÍNH MỚI CỦA ĐỀ TÀI

Đề tài đã nghiên cứu và hiện thực hóa thành công hệ thống ôn luyện thi trực tuyến **THPT Practice** trên nền tảng công nghệ hiện đại bao gồm Node.js, Express, Pug và cơ sở dữ liệu MongoDB. Những đóng góp kỹ thuật cốt lõi của đề tài được thể hiện qua các giải pháp xử lý nghiệp vụ cụ thể nhằm giải quyết các bài toán thực tiễn trong quản lý khảo thí.

Thứ nhất, hệ thống đã giải quyết triệt để bài toán bảo toàn tính vẹn toàn của dữ liệu lịch sử bài làm thông qua cơ chế lưu trữ bản sao dữ liệu (snapshot). Cụ thể, khi người dùng bắt đầu một phiên làm việc, hệ thống tự động trích xuất và lưu giữ toàn bộ bản sao nội dung câu hỏi, danh sách đáp án và phần giải thích chi tiết vào đối tượng lịch sử bài làm. Giải pháp này đảm bảo rằng mọi hoạt động chỉnh sửa hoặc xóa bỏ câu hỏi trong ngân hàng dữ liệu gốc của quản trị viên về sau hoàn toàn không làm sai lệch hay ảnh hưởng đến kết quả và lịch sử đánh giá năng lực của học sinh.

Thứ hai, đề tài xây dựng thành công cơ chế kiểm soát yêu cầu nộp bài đồng thời nhằm ngăn chặn xung đột dữ liệu. Bằng cách kết hợp điều kiện kiểm tra trạng thái phiên làm việc đang hoạt động trực tiếp trong thao tác cập nhật nguyên tử của cơ sở dữ liệu, hệ thống đảm bảo mỗi lượt làm bài chỉ được ghi nhận kết quả đúng một lần duy nhất, loại bỏ hoàn toàn hiện tượng ghi đè điểm số khi có lượng lớn người dùng gửi yêu cầu nộp bài vào cùng một thời điểm.

Các đóng góp trên đã được kiểm chứng thông qua quá trình kiểm thử chức năng toàn diện với mười hai kịch bản thực tế đạt kết quả chính xác tuyệt đối, cùng với quy trình đo tải thực nghiệm sử dụng công cụ k6 trên môi trường phần cứng tiêu chuẩn với quy mô lên tới một trăm người dùng ảo, qua đó khẳng định độ ổn định và tính khả thi khi triển khai trong thực tế.

6.2. HẠN CHẾ HIỆN TẠI VÀ HƯỚNG PHÁT TRIỂN

Bên cạnh những kết quả đạt được, hệ thống hiện tại vẫn tồn tại một số hạn chế nhất định cần được khắc phục trong các giai đoạn phát triển tiếp theo. Về mặt biên tập nội dung, công tác soạn thảo đề thi còn mang tính thủ công khi người dùng phải nhập liệu từng câu hỏi và tự lựa chọn thủ công khi cấu trúc đề, chưa hỗ trợ khả năng nhập hàng loạt dữ liệu từ các định dạng tệp phổ biến như Word hoặc Excel, cũng như chưa tích hợp tính năng tự động sinh đề thi theo ma trận kiến thức và mức độ khó. Về mặt bảo mật, khả năng chống gian lận trong quá trình làm bài chưa thực sự toàn diện đối với các hành vi tra cứu tài liệu hoặc sử dụng công cụ bên ngoài trên trình duyệt của người học. Ngoài ra, giao diện soạn thảo hỗ trợ công thức toán học qua KaTeX và tải ảnh qua Cloudinary nhưng vẫn còn những điểm bất tiện do chưa tích hợp công cụ vẽ trực tiếp đồ thị hoặc hình học chuyên sâu.

Hướng phát triển trong tương lai của dự án sẽ tập trung vào việc tự động hóa quy trình tạo đề thi thông qua thuật toán phân phối ma trận kiến thức, bổ sung tính năng nhập liệu hàng loạt, nâng cấp các biện pháp giám sát và chống gian lận trong phòng thi trực tuyến, đồng thời mở rộng các công cụ hỗ trợ trực quan hóa hình học nhằm đáp ứng tốt hơn nhu cầu ôn luyện đa dạng của học sinh.

6.3. KẾT LUẬN

Đề tài đã hoàn thành toàn bộ các mục tiêu đề ra trong việc thiết kế, xây dựng và vận hành hệ thống ôn luyện thi THPT trực tuyến. Ứng dụng đáp ứng trọn vẹn các yêu cầu nghiệp vụ từ quản lý tài khoản và phân quyền bảo mật bằng JSON Web Token lưu trong cookie httpOnly, quản lý ngân hàng câu hỏi đa dạng ba định dạng, tổ chức quy trình làm bài và chấm điểm tự động, quản lý nhóm học tập cho đến việc duy trì hệ thống thông báo thời gian thực thông qua giao thức WebSocket với thư viện Socket.IO. Thông qua việc áp dụng các mô hình kiến trúc chuẩn mực và giải pháp kỹ thuật tối ưu, dự án không chỉ cung cấp một công cụ học tập hữu ích cho học sinh và

giáo viên mà còn minh chứng năng lực hiện thực hóa các ứng dụng web hiệu suất cao bằng hệ sinh thái Node.js và MongoDB.

TÀI LIỆU THAM KHẢO

· Node.js Foundation. (2024). Node.js Documentation & Express 5 Framework Guide. Truy cập từ tài liệu kỹ thuật chính thức.

· MongoDB, Inc. (2024). MongoDB Manual and Mongoose ODM Documentation. Truy cập từ tài liệu cơ sở dữ liệu.

· Socket.IO. (2024). Real-time bidirectional event-based communication library documentation. Truy cập từ tài liệu kỹ thuật Socket.IO.

· Auth0. (2024). JSON Web Tokens (JWT) specification, jsonwebtoken and cookie-parser libraries. Truy cập từ kho lưu trữ mã nguồn mở.

· Quill & KaTeX Contributors. (2024). Quill Rich Text Editor and KaTeX Mathematical Formatting Documentation. Truy cập từ tài liệu thư viện giao diện.

· Cloudinary. (2024). Cloudinary API and Multer Integration Guide for Media Management. Truy cập từ tài liệu dịch vụ đám mây.

· Grafana Labs. (2024). Grafana k6 Load Testing Tool Documentation and Performance Scripting. Truy cập từ tài liệu kiểm thử tải.

PHỤ LỤC

PHỤ LỤC A: SCHEMA CƠ SỞ DỮ LIỆU RÚT GỌN

Hệ thống quản lý dữ liệu thông qua các collection chính trên cơ sở dữ liệu MongoDB:

accounts: Lưu trữ thông tin định danh tài khoản, mật khẩu đã mã hóa bằng bcrypt và vai trò phân quyền người dùng hoặc quản trị viên.

subjects và topics: Quản lý danh mục phân loại môn học và các chuyên đề ôn tập tương ứng.

questions: Lưu trữ ngân hàng câu hỏi bao gồm ba dạng trắc nghiệm, đúng/sai và điền đáp án, tích hợp nội dung định dạng giàu thông tin và công thức KaTeX.

exams: Cấu trúc đề thi bao gồm danh sách câu hỏi, thời lượng và thông tin cấu hình.

attempts: Lưu trữ lịch sử bài làm của học sinh, bao gồm mảng questionSnapshotSchema ghi nhận bản sao câu hỏi tại thời điểm bắt đầu và mảng answerSchema ghi nhận đáp án, điểm số chi tiết.

groups: Quản lý danh sách nhóm học tập, thành viên tham gia và cơ chế chia sẻ đề thi trong nhóm.

notifications: Lưu trữ hệ thống thông báo sự kiện phục vụ tính năng truyền tải thời gian thực.

latexes: Quản lý kho công thức toán học hỗ trợ soạn thảo.

PHỤ LỤC B: CẤU HÌNH MÔI TRƯỜNG (.ENV MẪU)

Tệp cấu hình môi trường được thiết lập tại thư mục gốc với các thông số kết nối cơ sở dữ liệu và dịch vụ bên thứ ba:

MONGO_URI=mongodb://127.0.0.1:27017/THPT

JWT_SECRET=replace_with_a_long_random_secret

CLOUDINARY_CLOUD_NAME=your_cloud_name

CLOUDINARY_API_KEY=your_api_key

CLOUDINARY_API_SECRET=your_api_secret

PHỤ LỤC C: TÓM TẮT KỊCH BẢN KIỂM THỬ TẢI K6 (EXAMS.JS)

Kịch bản kiểm thử tải thực hiện phương thức truy vấn GET /exams trên ứng dụng với các mức tải quy định gồm 10, 50 và 100 người dùng ảo (VUs) trong khoảng thời gian chạy 60 giây cho mỗi mức tải. Script thực hiện kiểm tra mã trạng thái HTTP 200 và tính nguyên vẹn của nội dung phản hồi HTML, ghi nhận các chỉ số hiệu năng thời gian phản hồi ở mức phân vị p50 và p95 phục vụ đánh giá độ bão hòa của hệ thống.